DAY 1

Types of Data:-

1. Unstructured Data

- Data not organized in a predefined format.
- Usually hard to read, store, or process.
- Often lacks meaning without context.

Example:

201 Lakemba SYD NSW AUS

2. Structured Data

- Organized and stored in a predefined format like rows and columns.
- Easy to read and process.
- Has clear meaning and organization.

Example:

Unit	Locality	City	State	Country
201	Lakemba	SYD	NSW	AUS

Oracle Database Overview

- **Oracle** is a Relational Database Management System (RDBMS) used to store structured data.
- Developed by **Oracle Corporation**.

Oracle Database Components

Oracle

├── SQL (Structured Query Language)

└── PL/SQL (Procedural Language/SQL)

Client-Server Architecture

Frontend

Interface visible to the user for performing operations.
Examples: Java, C++, C#, Python, .NET

Communication Channels

Middleware that connects frontend to backend.

Examples:

- JDBC (Java Database Connectivity)
- ODBC (Open Database Connectivity)
- PDBC (Python Database Connectivity)

Backend

The hidden part that handles logic and data operations.
Examples: Oracle, MySQL, MongoDB, NoSQL

Database Management System (DBMS)

- **DBMS**: Database + Software to manage data.
- **Activities:**
    - Add data
    - Retrieve data
    - Update data
    - Delete data

Definition:- A **Database Management System (DBMS)** is a software system that allows users to define, create, manage, and control access to a database. It acts as an interface between the database and the end users or applications.

RDBMS (Relational DBMS)

- A type of DBMS based on relational theory.
- Stores data in **tables (relations)**.

Examples: Oracle, MySQL, SQL Server, PostgreSQL, Teradata, Sybase

Definition:- An **RDBMS (Relational Database Management System)** is a type of DBMS that stores data in the form of **related tables** (also known as **relations**). It uses **Structured Query Language (SQL)** for accessing and managing data efficiently.

DBMS vs RDBMS – Interview Question

DBMS	RDBMS
Database Management System	Relational Database Management System
Stores data in files	Stores data in tables
Supports single user	Supports multiple users
No normalization	Supports normalization
Less secure	More secure

DAY 2

SQL – Structured Query Language

- Pronounced as **SEQUEL**.
- Command-based, case-insensitive language.
- Developed by **E.F. Codd** at **IBM (1972)**.

Examples of Commands:

SELECT * FROM student;

select * from student;

SQL Sub-languages

Sub-language	Purpose	Commands
DDL (Data Definition Language)	Define schema/structure	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML (Data Manipulation Language)	Manage data in tables	INSERT, UPDATE, DELETE, MERGE
DRL/DQL (Data Retrieval/Query Language)	Fetch data	SELECT
TCL (Transaction Control Language)	Control transactions	COMMIT, ROLLBACK, SAVEPOINT
DCL (Data Control Language)	Control access	GRANT, REVOKE

DDL – Data Definition Language

Definition:- DDL (Data Definition Language) is a subset of SQL used to define and manage the **structure or schema** of database objects like **tables, views, indexes, schemas**, etc.
It does **not manipulate data**, but rather defines how the data is stored.

Key Points

- Executes structural changes to the database.
- DDL commands are **auto-committed**—once executed, changes cannot be rolled back.
- Mainly used during **database design** and **schema modification** stages.

Common DDL Commands

Command	Description
CREATE	Used to create new database objects such as tables, views, or schemas.
ALTER	Used to modify existing database objects (e.g., add/remove columns in a table).
DROP	Used to remove existing objects from the database permanently.
TRUNCATE :-	Deletes all rows from a table but retains its structure for future use.
RENAME	Changes the name of a table or column.

DML – Data Manipulation Language

Definition:-DML (Data Manipulation Language) is a subset of SQL used to **manage and manipulate data** stored in database tables. It allows users to **insert**, **modify**, **delete**, and **merge** records.

◈ DML operates on the data within the structure defined by DDL.

Key Characteristics

- Affects **only the data**, not the table structure.
- DML commands can be **rolled back or committed** using **TCL commands**.
- Ensures **data integrity** when used with constraints and transactions.

Common DML Commands

Command		Purpose
INSERT	:	Adds new rows of data into a table.
UPDATE	:	Modifies existing data in one or more columns of a row.
DELETE	:	Removes one or more rows from a table based on a condition.
MERGE	:	Performs both insert and update in one SQL command.

DRL / DQL – Data Retrieval / Query Language

Definition:-DRL (Data Retrieval Language) or **DQL (Data Query Language)** is a subset of SQL used to **fetch or retrieve data** from the database.
The most commonly used command in this category is the **SELECT** statement.

- DRL does **not modify** data—it only **reads or queries** data stored in tables.

Key Features

- Retrieves **specific columns**, **rows**, or **entire tables**.
- Supports filtering, sorting, grouping, and joining data.
- Does **not require commit or rollback**, as it does not change data.

Primary Command: SELECT

Command Purpose

SELECT	Used to fetch data from one or more tables.

TCL – Transaction Control Language

Definition:-TCL (Transaction Control Language) is used to manage transactions in a database.
A transaction is a **sequence of operations** performed as a single logical unit of work. TCL ensures **data consistency**, **integrity**, and **reliability** during these operations.

Key Characteristics

- A **transaction** ends when a **COMMIT** or **ROLLBACK** is issued.
- Ensures **ACID properties** (Atomicity, Consistency, Isolation, Durability).
- Often used in combination with **DML** commands.

TCL Commands

Command	Purpose
COMMIT :	Saves all changes made in the transaction permanently.
ROLLBACK :	Reverts all changes made in the current transaction.
SAVEPOINT :	Sets a point to which you can rollback later.

DCL – Data Control Language

Definition:-DCL (Data Control Language) is used to control access to data in the database. It helps enforce data security and permissions.

DCL Commands

Command	Purpose
GRANT	Provides user access privileges to database objects.
REVOKE	Removes user access privileges from database objects.

Table in Oracle

- A table stores data in **rows and columns**.

Example Table:

SNO	SNAME	SADD
101	Alan	USA
102	Jose	UAE
103	Lisa	UK

Oracle is **case-insensitive** for queries, but **case-sensitive** for data.

Connecting to Oracle Database

To Connect:

SQL> connect

Username: system

Password: admin

or

SQL> conn system/admin

To Disconnect:

SQL> disconnect

or

SQL> disc

1. CREATE Command

Purpose:
Used to create a new table in the database.

Syntax:

```
CREATE TABLE <table_name> (
  column1 datatype(size),
  column2 datatype(size),
  ...,
  columnN datatype(size)
);
```

Examples:

```
CREATE TABLE student (
  sno NUMBER(3),
  sname VARCHAR2(10),
  sadd VARCHAR2(12)
);

CREATE TABLE emp (
  eid NUMBER(3),
  ename VARCHAR2(10),
  esal NUMBER(10,2),
  deptno NUMBER(3),
  job VARCHAR2(10),
  comm NUMBER(6)
);

CREATE TABLE dept (
  deptno NUMBER(3),
  dname VARCHAR2(10),
  dloc VARCHAR2(12)
);
```

---

2. DESCRIBE Command

Purpose:
View the structure of an existing table.

Syntax:

DESC <table_name>;

Examples:

DESC emp;

DESC student;

DESC dept;

---

3. INSERT Command

Purpose:
Insert a new row into a table.

Syntax:

INSERT INTO <table_name> VALUES(val1, val2, ..., valN);

Examples:

INSERT INTO student VALUES (101, 'raja', 'hyd');

INSERT INTO student VALUES (102, 'ravi', NULL);

Notes:

- NULL represents undefined or missing data.
- You can specify columns explicitly:

INSERT INTO student (sno, sname, sadd) VALUES (103, 'ramana', 'vizag');

INSERT INTO student (sno, sname) VALUES (104, 'ramulu');

Dynamic input with substitution variables:

INSERT INTO student VALUES (&sno, '&sname', '&sadd');

---

4. COMMIT Command

Purpose:
Make all pending changes permanent in the database.

Example:

COMMIT;

---

5. SELECT Command

Purpose:
Retrieve records from the database.

Syntax:

SELECT * FROM <table_name>;

Examples:

SELECT * FROM emp;

SELECT sno, sname FROM student;

SELECT sno + 100 AS ROLLNO FROM student;

Notes:

- Selecting specific columns is called **projection**.
- You can use arithmetic expressions and column aliases to improve readability.

---

6. WHERE Clause

Purpose:
Filter records based on a condition.

Syntax:

SELECT * FROM <table_name> WHERE <condition>;

Examples:

SELECT * FROM student WHERE sno = 101;

SELECT * FROM emp WHERE deptno = 10;

SELECT * FROM emp WHERE comm IS NULL;

---

7. UPDATE Command

Purpose:
Modify existing records in a table.

Syntax:

UPDATE <table_name> SET column = value WHERE condition;

Examples:

UPDATE student SET sname = 'rani' WHERE sno = 105;

UPDATE emp SET job = 'Salesman' WHERE job = 'Clerk';

UPDATE emp SET esal = esal + 1000 WHERE eid = 201;

Important:

- Omitting the WHERE clause updates all rows.

---

8. DELETE Command

Purpose:
Remove records from a table.

Syntax:

DELETE FROM <table_name> WHERE condition;

Examples:

DELETE FROM student WHERE sno = 101;

DELETE FROM emp WHERE esal = 30000;

Important:

- Omitting the WHERE clause deletes all rows in the table.

Additional Notes

- All **DML commands** (INSERT, UPDATE, DELETE) are **temporary** until a COMMIT is issued.
- Always use **WHERE clause** carefully in UPDATE and DELETE to avoid unwanted changes.

Sample Tables and Data

Department Table:

```
CREATE TABLE dept (
  deptno NUMBER(3),
  dname VARCHAR2(10),
  dloc VARCHAR2(12)
);

INSERT INTO dept VALUES (10, 'CSE', 'HYD');
INSERT INTO dept VALUES (20, 'EEE', 'DELHI');
INSERT INTO dept VALUES (30, 'ECE', 'PUNE');
INSERT INTO dept VALUES (40, 'MEC', 'VIZAG');
COMMIT;
```

Employee Table:

```
CREATE TABLE emp (
  eid NUMBER(3),
  ename VARCHAR2(10),
  esal NUMBER(10,2),
  deptno NUMBER(3),
  job VARCHAR2(10),
  comm NUMBER(6)
);
INSERT INTO emp VALUES (201, 'Alan', 9000, 10, 'Clerk', NULL);
INSERT INTO emp VALUES (202, 'Jose', 18000, 10, 'Clerk', 100);
INSERT INTO emp VALUES (203, 'Kelvin', 29000, 20, 'HR', 200);
INSERT INTO emp VALUES (204, 'Lisa', 19000, 20, 'HR', 500);
INSERT INTO emp VALUES (205, 'Nelson', 49000, 30, 'Manager', 300);
INSERT INTO emp VALUES (206, 'Jack', 30000, 30, 'Manager', 900);
COMMIT;
```

Common Interview Queries

Query Purpose	SQL Query
Display database schema name	SELECT * FROM global_name;
List all tables	SELECT * FROM tab;
Retrieve all employee records	SELECT * FROM emp;
Retrieve employee ID, name, and salary	SELECT eid, ename, esal FROM emp;
Retrieve employee salary and annual salary	SELECT eid, ename, esal, esal * 12 AS ANNUAL_SALARY FROM emp;
Display employees in department 10	SELECT * FROM emp WHERE deptno = 10;
Display employees with job title 'Manager'	SELECT * FROM emp WHERE job = 'Manager';
Display employees with no commission	SELECT * FROM emp WHERE comm IS NULL;
Display employees with salary greater than 15000	SELECT * FROM emp WHERE esal > 15000;

DAY 3

Operators

AND Operator

Purpose:
The AND operator is used to combine multiple conditions. It returns records only when **both conditions** are true.

Example Syntax:

SELECT * FROM <table_name> WHERE condition1 AND condition2;

Examples:

SELECT * FROM student WHERE sno = 101 AND sname = 'raja'; -- Returns the record where both conditions are true

SELECT * FROM student WHERE sno = 101 AND sname = 'ravi'; -- No rows selected, as both conditions are false

---

2. OR Operator

Purpose:
The OR operator returns records if **any of the conditions** is true.

Example Syntax:

SELECT * FROM <table_name> WHERE condition1 OR condition2;

Examples:

SELECT * FROM student WHERE sno = 101 OR sname = 'raja'; -- Returns records where either condition is true

SELECT * FROM student WHERE sno = 101 OR sname = 'ravi'; -- Returns records where either condition is true

---

3. NOT Operator

Purpose:
The NOT operator is used to return records where the specified condition is **not true**. The <> symbol is equivalent to NOT.

Example Syntax:

SELECT * FROM <table_name> WHERE NOT condition;

SELECT * FROM <table_name> WHERE column <> value;

Examples:

SELECT * FROM student WHERE NOT sno = 101; -- Returns records where sno is not 101

SELECT * FROM student WHERE sno <> 101;   -- Equivalent to the above query

SELECT * FROM student WHERE NOT sname = 'raja'; -- Excludes records where sname is 'raja'

SELECT * FROM student WHERE sname <> 'raja'; -- Equivalent to the above query

---

4. IN Operator

Purpose:
The IN operator is a shorthand for multiple OR conditions. It returns records where the value of a column matches any value in a given list.

Example Syntax:

SELECT * FROM <table_name> WHERE column IN (value1, value2, ..., valueN);

Examples:

SELECT * FROM student WHERE sno IN (101, 102, 103, 108); -- Returns records where sno matches any of the values in the list

SELECT * FROM student WHERE sname IN ('raja', 'ravi', 'rakesh'); -- Returns records where sname matches any of the specified names

---

5. BETWEEN Operator

Purpose:
The BETWEEN operator is used to return records where the column values lie within a **range** of values. It internally uses the AND operator.

Example Syntax:

SELECT * FROM <table_name> WHERE column BETWEEN value1 AND value2;

Examples:

SELECT * FROM student WHERE sno BETWEEN 101 AND 105; -- Returns records where sno is between 101 and 105

SELECT * FROM emp WHERE deptno BETWEEN 10 AND 30; -- Returns records where deptno is between 10 and 30

Interview Queries

1. **Display employee information whose employee ID is 204 and job is HR:**

SELECT * FROM emp WHERE eid = 204 AND job = 'HR';

2. **Display employee information for employees working in departments 10, 20, and 30:**

SELECT * FROM emp WHERE deptno IN (10, 20, 30);

3. **Display employee information for employees with a salary greater than 35000 and less than 50000:**

SELECT * FROM emp WHERE esal BETWEEN 35000 AND 50000;

-- Alternative:

SELECT * FROM emp WHERE esal >= 35000 AND esal <= 50000;

4. **Display employee information for employees who are not working in department 10:**

SELECT * FROM emp WHERE NOT deptno = 10;

-- Alternative:

SELECT * FROM emp WHERE deptno <> 10;

Summary of Logical Operators

Operator	Description	Example
AND	Returns records where both conditions are true	SELECT * FROM student WHERE sno = 101 AND sname = 'raja';
OR	Returns records where any condition is true	SELECT * FROM student WHERE sno = 101 OR sname = 'raja';

Operator	Description	Example
NOT	Returns records where the condition is not true	SELECT * FROM student WHERE NOT sno = 101;
IN	Returns records where the column value matches any value in the list	SELECT * FROM student WHERE sno IN (101, 102, 103);
BETWEEN	Returns records where the column value is within a range	SELECT * FROM student WHERE sno BETWEEN 101 AND 105;

Pattern Matching Operators

Pattern matching operators are used to filter data based on specific patterns using the LIKE keyword. There are two types:

1. **Percentage (%)** – Represents zero or more characters.
2. **Underscore (_)** – Represents a single character.

Examples using Percentage (%)

- Employees whose name starts with 'A':

```
SELECT * FROM emp WHERE ename LIKE 'A%';
```

- Employees whose name ends with 'n':

```
SELECT * FROM emp WHERE ename LIKE '%n';
```

- Employees whose name contains 'l':

```
SELECT * FROM emp WHERE ename LIKE '%l%';
```

Examples using Underscore (_)

- Second letter is 'l':

```
SELECT * FROM emp WHERE ename LIKE '_l%';
```

- Second last letter is 'a':

```
SELECT * FROM emp WHERE ename LIKE '%a_';
```

SQL Comments

- Single-line comment:

-- This is a comment

Creating Duplicate Tables

You can create a copy of an existing table using CREATE TABLE AS SELECT:

CREATE TABLE employee AS SELECT * FROM emp;

CREATE TABLE employee AS SELECT eid, ename, esal FROM emp;

CREATE TABLE employee AS SELECT * FROM emp WHERE deptno = 10;

CREATE TABLE employee AS SELECT * FROM emp WHERE comm IS NULL;

CREATE TABLE employee AS SELECT * FROM emp WHERE eid IN (201, 202, 203);

CREATE TABLE employee AS SELECT * FROM emp WHERE eid BETWEEN 201 AND 206;

Command: CL SCR

Clears the SQL command screen (used in SQL*Plus or command-line tools).

CL SCR

DDL Commands

Command	Description
CREATE	Creates tables
ALTER	Modifies columns
DROP	Deletes tables
TRUNCATE	Deletes all records permanently
RENAME	Renames a table or column

ALTER Command

Used to perform the following:

1. Add new columns
2. Modify existing columns
3. Drop columns
4. Rename columns

Add Columns

ALTER TABLE student ADD (state VARCHAR2(10));

ALTER TABLE student ADD (pincode NUMBER(8));

ALTER TABLE student ADD (state VARCHAR2(10), pincode NUMBER(8));

Modify Columns

ALTER TABLE student MODIFY (state VARCHAR2(15));

ALTER TABLE student MODIFY (pincode VARCHAR2(8));

Drop Columns

ALTER TABLE student DROP (state, pincode);

Rename Columns

ALTER TABLE student RENAME COLUMN sadd TO city;

ALTER TABLE emp RENAME COLUMN esal TO dailywages;

ALTER TABLE emp RENAME COLUMN job TO designation;

DROP Command

Deletes a table permanently from the database:

DROP TABLE emp;

DROP TABLE student;

DROP TABLE dept;

TRUNCATE Command

Removes all records from a table permanently:

TRUNCATE TABLE emp;

TRUNCATE TABLE dept;

TRUNCATE TABLE student;

DELETE vs TRUNCATE

DELETE	TRUNCATE
Deletes data temporarily	Deletes data permanently
Can rollback	Cannot rollback
Can use WHERE clause	Cannot use WHERE clause
DML Command	DDL Command

RENAME Command

Renames a table:

RENAME emp TO employees;

RENAME dept TO department;

RENAME student TO students;

DAY 4

SQL Functions

Functions are used to manipulate data. They are categorized as:

1. **Group Functions** (Multiple Row Functions)
2. **Scalar Functions** (Single Row Functions)

Group Functions

Function	Description
SUM()	Total value of a column
AVG()	Average value
MAX()	Maximum value
MIN()	Minimum value
COUNT(*)	Number of rows including NULLs
COUNT(expr)	Number of non-NULL values

Examples:

SELECT SUM(esal) FROM emp;

SELECT AVG(esal) FROM emp;

SELECT MAX(esal) FROM emp;

SELECT MIN(esal) FROM emp;

SELECT COUNT(*) FROM emp;

SELECT COUNT(eid) FROM emp;

SELECT COUNT(comm) FROM emp;

User Authentication Example

```
DROP TABLE userlist;

CREATE TABLE userlist (uname VARCHAR2(10), pwd VARCHAR2(10));

INSERT INTO userlist VALUES ('raja', 'rani');

INSERT INTO userlist VALUES ('king', 'kingdom');

COMMIT;
```

Check login:

```
SELECT COUNT(*) FROM userlist WHERE uname = 'raja' AND pwd = 'rani'; -- 1

SELECT COUNT(*) FROM userlist WHERE uname = 'raja' AND pwd = 'rani1'; -- 0
```

DUAL Table

A dummy table in Oracle used for calculations or functions:

```
SELECT 10+20 FROM dual;

SELECT 10*30 FROM dual;

SELECT SYSDATE FROM dual;

SELECT CURRENT_DATE FROM dual;
```

Scalar Functions

Operate on a single row and return one result.

i) Character Functions

Function	Description
UPPER()	Converts to uppercase
LOWER()	Converts to lowercase
INITCAP()	Capitalizes first letter of each word
LPAD()	Left-pads a string
RPAD()	Right-pads a string

LTRIM()	Trims characters from left
RTRIM()	Trims characters from right
TRIM()	Trims characters from both sides

Examples:

```
SELECT UPPER('oracle') FROM dual;
SELECT LOWER('ORACLE') FROM dual;
SELECT INITCAP('oracle training') FROM dual;
SELECT LPAD('oracle', 10, 'z') FROM dual;
SELECT RPAD('oracle', 10, 'z') FROM dual;
SELECT LTRIM('zzoraclezz', 'z') FROM dual;
SELECT RTRIM('zzoraclezz', 'z') FROM dual;
SELECT TRIM('z' FROM 'zzoraclezz') FROM dual;
```

---

ii) Number Functions

Function	Description
ABS()	Absolute value
POWER(a,b)	a raised to the power of b
CEIL()	Next highest integer
FLOOR()	Next lowest integer
ROUND()	Rounds to nearest integer
TRUNC()	Truncates decimal part
GREATEST()	Returns greatest value
LEAST()	Returns least value

Examples:

```
SELECT ABS(-56) FROM dual;

SELECT POWER(2, 3) FROM dual;

SELECT CEIL(10.6) FROM dual;

SELECT FLOOR(10.6) FROM dual;

SELECT ROUND(10.5) FROM dual;

SELECT TRUNC(10.56) FROM dual;

SELECT GREATEST(10, 20, 30) FROM dual;

SELECT LEAST(10, 20, 30) FROM dual;
```

Working with Date Values

- Different DBMSs support different date formats.
    - Oracle: dd-MMM-yy
    - MySQL: yyyy-MM-dd

Example:

```
CREATE TABLE emp1 (eid NUMBER(3), ename VARCHAR2(10), edoj DATE);

INSERT INTO emp1 VALUES (301, 'Alan', '01-JAN-22');

INSERT INTO emp1 VALUES (302, 'Jose', SYSDATE);

INSERT INTO emp1 VALUES (303, 'Lisa', CURRENT_DATE);

INSERT INTO emp1 VALUES (304, 'Jessi', '15-MAR-23');

COMMIT;
```

iii) Date Functions

Function	Description
ADD_MONTHS()	Adds months to a date
MONTHS_BETWEEN()	Finds months between two dates
NEXT_DAY()	Finds next given weekday after a date
LAST_DAY()	Finds last date of a month

Examples:

SELECT ADD_MONTHS(SYSDATE, 4) FROM dual;

SELECT MONTHS_BETWEEN('01-JAN-23', '01-JUL-23') FROM dual;

SELECT NEXT_DAY(SYSDATE, 'SUNDAY') FROM dual;

SELECT LAST_DAY(SYSDATE) FROM dual;

---

iv) Conversion Functions

Used to convert data from one type to another.

TO_CHAR() - Number Formatting

SELECT TO_CHAR(esal, '9,999') FROM emp;

SELECT TO_CHAR(esal, '$99,999') AS ESAL FROM emp;

TO_CHAR() - Date Formatting

SELECT TO_CHAR(SYSDATE, 'dd-MM-yyyy') FROM dual;

SELECT TO_CHAR(SYSDATE, 'day') FROM dual;

SELECT TO_CHAR(SYSDATE, 'month') FROM dual;

SELECT TO_CHAR(SYSDATE, 'YYYY-MM-dd HH:MI:SS') FROM dual;

DAY 5

Group By Clause

Used to group rows to apply group functions.

Examples:

SELECT SUM(esal), deptno FROM emp GROUP BY deptno;

SELECT MAX(esal), job FROM emp GROUP BY job;

SELECT AVG(esal), deptno FROM emp GROUP BY deptno;

Having Clause

Used to filter grouped records.

Examples:

SELECT SUM(esal), deptno FROM emp GROUP BY deptno HAVING SUM(esal) > 35000;

SELECT MIN(esal), job FROM emp GROUP BY job HAVING MIN(esal) < 15000;

ORDER BY Clause

Used to sort query results.

Examples:

SELECT * FROM emp ORDER BY eid;

SELECT * FROM emp ORDER BY ename;

SELECT * FROM emp ORDER BY esal DESC;

Integrity Constraints

Rules to enforce data accuracy and integrity in a table.

Constraint	Description
NOT NULL	Disallows NULL values
UNIQUE	Disallows duplicate values
PRIMARY KEY	Combines NOT NULL + UNIQUE
FOREIGN KEY	Links to a primary key in another table
CHECK	Validates data based on a condition

Constraints can be created at two levels:

- Column level
- Table level

Constraints at Column Level

- Constraint is defined right after the column data type in the column definition.
- Applies directly to that column only.
- Usually used for simple constraints on a single column.

Constraints at Table Level

- Constraints are defined after all the columns are listed.
- Useful when the constraint involves multiple columns (like composite primary key) or for better readability.
- Applies to the whole table or multiple columns.

---

1) NOT NULL Constraint

- Purpose: Does not accept NULL values.
- Allows duplicate values.
- Usually created at column level.

Example: NOT NULL at Column Level

```
DROP TABLE student;
CREATE TABLE student (
  sno NUMBER(3) NOT NULL,
  sname VARCHAR2(10),
  sadd VARCHAR2(12)
);

INSERT INTO student VALUES (101, 'raja', 'hyd');       -- valid
INSERT INTO student VALUES (NULL, 'ravi', 'delhi');    -- invalid, sno cannot be NULL
INSERT INTO student VALUES (101, 'ramana', 'vizag');   -- valid (duplicates allowed)
COMMIT;
```

NOT NULL for multiple columns:

```
DROP TABLE student;

CREATE TABLE student (
  sno NUMBER(3) NOT NULL,
  sname VARCHAR2(10) NOT NULL,
  sadd VARCHAR2(12) NOT NULL
);

INSERT INTO student VALUES (101, 'raja', 'hyd');       -- valid
INSERT INTO student VALUES (NULL, 'ravi', 'delhi');    -- invalid
INSERT INTO student VALUES (102, NULL, 'vizag');       -- invalid
INSERT INTO student VALUES (103, 'ramana', NULL);      -- invalid
COMMIT;
```

---

2) **UNIQUE Constraint**

- Purpose: Does not accept duplicate values.
- Allows NULL values.
- Can be created at column level or table level.

UNIQUE Constraint at Column Level

```
DROP TABLE student;
CREATE TABLE student (
  sno NUMBER(3) UNIQUE,
  sname VARCHAR2(10),
  sadd VARCHAR2(12)
);

INSERT INTO student VALUES (101, 'raja', 'hyd');       -- valid
INSERT INTO student VALUES (NULL, 'ravi', 'delhi');    -- valid (NULL allowed)
INSERT INTO student VALUES (101, 'ramana', 'vizag');   -- invalid (duplicate sno)

COMMIT;
```

UNIQUE Constraint at Table Level

```
DROP TABLE student;
CREATE TABLE student (
  sno NUMBER(3),
  sname VARCHAR2(10),
  sadd VARCHAR2(12),
  UNIQUE(sno)
);

INSERT INTO student VALUES (101, 'raja', 'hyd');       -- valid
```

```
INSERT INTO student VALUES (NULL, 'ravi', 'delhi');    -- valid
INSERT INTO student VALUES (101, 'ramana', 'vizag');   -- invalid
COMMIT;
```

UNIQUE Constraint on Multiple Columns (Column Level)

```
DROP TABLE student;
CREATE TABLE student (
  sno NUMBER(3) UNIQUE,
  sname VARCHAR2(10) UNIQUE,
  sadd VARCHAR2(12) UNIQUE
);
INSERT INTO student VALUES (101, 'raja', 'hyd');        -- valid
INSERT INTO student VALUES (102, 'ravi', 'delhi');      -- valid
INSERT INTO student VALUES (101, 'ramana', 'vizag');    -- invalid (duplicate sno)
COMMIT;
```

---

3) PRIMARY KEY Constraint

- Combines NOT NULL + UNIQUE constraints.
- Does not accept NULL or duplicate values.
- A table can have only one primary key.
- Can be created at column level or table level.

PRIMARY KEY at Column Level

```
DROP TABLE student;
CREATE TABLE student (
  sno NUMBER(3) PRIMARY KEY,
  sname VARCHAR2(10),
  sadd VARCHAR2(12)
);
```

```
INSERT INTO student VALUES (101, 'raja', 'hyd');       -- valid
INSERT INTO student VALUES (NULL, 'ravi', 'delhi');    -- invalid (NULL not allowed)
INSERT INTO student VALUES (101, 'ramana', 'vizag');   -- invalid (duplicate sno)
COMMIT;
```

PRIMARY KEY at Table Level

```
DROP TABLE student;
CREATE TABLE student (
  sno NUMBER(3),
  sname VARCHAR2(10),
  sadd VARCHAR2(12),
  PRIMARY KEY (sno)
);

INSERT INTO student VALUES (101, 'raja', 'hyd');       -- valid
INSERT INTO student VALUES (NULL, 'ravi', 'delhi');    -- invalid
INSERT INTO student VALUES (101, 'ramana', 'vizag');   -- invalid
COMMIT;
```

Foreign Key

- Establishes a relationship between two tables (parent & child).
- **Parent table:** Must have Primary Key or Unique Key.
- **Child table:** Has Foreign Key referencing the parent table's PK.
- Foreign Key accepts only values present in parent’s PK column.
- Foreign Key values can be duplicated or null.
- Foreign Key name may differ from Primary Key name but **data types must match**.
- To drop tables: **Drop child table first, then parent table.**

Example:

```
-- Parent table
CREATE TABLE college (
  sno NUMBER(3) PRIMARY KEY,
  sname VARCHAR2(10),
  sadd VARCHAR2(12)
);

INSERT INTO college VALUES (101, 'raja', 'hyd');
INSERT INTO college VALUES (102, 'ravi', 'delhi');
INSERT INTO college VALUES (103, 'ramana', 'vizag');
COMMIT;

-- Child table
CREATE TABLE library (
  roll_no NUMBER(3) REFERENCES college(sno),
  book_name VARCHAR2(10)
);

INSERT INTO library VALUES (101, 'Java');
INSERT INTO library VALUES (102, 'Oracle');
INSERT INTO library VALUES (103, 'React');
INSERT INTO library VALUES (103, 'Spring');
INSERT INTO library VALUES (NULL, 'HTML');
INSERT INTO library VALUES (104, 'Testing'); -- Invalid, 104 not in college.sno
COMMIT;
-- Drop order:
DROP TABLE library;
DROP TABLE college;
```

Check Constraint

- Ensures column values satisfy specified condition (domain constraint).
- Can be created at **column level** or **table level**.

Column-level examples:

```
DROP TABLE student;
CREATE TABLE student (
  sno NUMBER(3),
  sname VARCHAR2(10),
  smarks NUMBER(3) CHECK(smarks <= 100)
);
-- Valid: smarks <= 100
-- Invalid: smarks > 100

CREATE TABLE student (
  sno NUMBER(3),
  sname VARCHAR2(10),
  smarks NUMBER(3) CHECK(smarks BETWEEN 0 AND 100)
);
CREATE TABLE student (
  sno NUMBER(3),
  sname VARCHAR2(10) CHECK(sname = UPPER(sname)),
  smarks NUMBER(3)
);
CREATE TABLE student (
  sno NUMBER(3),
  sname VARCHAR2(10) CHECK(sname = LOWER(sname)),
  smarks NUMBER(3)
);
```

Table-level example:

```
CREATE TABLE student (
  sno NUMBER(3),
  sname VARCHAR2(10),
  smarks NUMBER(3),
  CHECK (sname = LOWER(sname))
);
```

Alter Table Constraint Queries

- Add primary key:

```
ALTER TABLE emp ADD PRIMARY KEY(eid);
```

- Drop primary key:

```
ALTER TABLE emp DROP PRIMARY KEY;
```

DAY 6

Difference between ROWNUM and ROWID

Feature	ROWNUM	ROWID
Meaning	Temporary sequential number assigned to rows in result set	Physical address of the row in database storage
Start value	Always starts at 1 and increments by 1	Unique permanent identifier
Persistence	Exists only during query execution	Permanent for the life of the row
Use case	To limit number of rows in query result	To uniquely identify a row

Examples:

SELECT eid, ename, esal FROM emp;

SELECT ROWNUM, eid, ename, esal FROM emp;

SELECT ROWID, ROWNUM, eid, ename, esal FROM emp;

Interview Query Examples

- First 3 records:

SELECT * FROM emp WHERE ROWNUM <= 3;

- 4th record:

SELECT * FROM emp WHERE ROWNUM <= 4

MINUS

SELECT * FROM emp WHERE ROWNUM <= 3;

- 6th record:

SELECT * FROM emp WHERE ROWNUM <= 6

MINUS

SELECT * FROM emp WHERE ROWNUM <= 5;

TCL Commands

1) COMMIT

- Makes changes permanent in the database.

```
COMMIT;
```

2) ROLLBACK

- Undo uncommitted changes.

```
ROLLBACK;
```

3) SAVEPOINT

- Create a marker to rollback partially.

```
SAVEPOINT sp1;

ROLLBACK TO sp1;
```

Example for SAVEPOINT:

```
INSERT INTO student VALUES (101, 'raja', 'hyd');

INSERT INTO student VALUES (102, 'ravi', 'delhi');

SAVEPOINT sp1;


INSERT INTO student VALUES (103, 'john', 'USA');

INSERT INTO student VALUES (104, 'hendry', 'UK');

SAVEPOINT sp2;


INSERT INTO student VALUES (105, 'Jerry', 'Japan');

INSERT INTO student VALUES (106, 'Chan', 'China');

ROLLBACK TO sp2; -- removes 105, 106
```

DAY 7

DCL Commands

Schema

- A **schema** is a memory location where SQL commands are executed.

Privileges

- Permissions or rights given to users are called **privileges**.
- Two types of privileges:
    1. **System Privilege** — Given by DBA to a user (e.g., CREATE SESSION, CREATE TABLE).
    2. **Object Privilege** — Given by one user to another user on database objects (e.g., SELECT, INSERT on a table).

GRANT

- Used to **give permissions** to a user.
- **Syntax:**

GRANT privilege1, privilege2 TO <user_name>;

REVOKE

- Used to **remove permissions** from a user.
- **Syntax:**

REVOKE privilege1, privilege2 FROM <user_name>;

Example Workflow for User Privileges

```
-- Create users
CREATE USER anil IDENTIFIED BY anil123;
CREATE USER anjali IDENTIFIED BY anjali;

-- Initially, logins denied
-- DBA grants basic privileges
GRANT connect, resource TO anil, anjali;

-- Users can now login
CONNECT anil/anil123;
CONNECT anjali/anjali;

-- User anil creates and inserts into table
CREATE TABLE employee (
  eid NUMBER(3),
  ename VARCHAR2(10),
  esal NUMBER(8)
);

INSERT INTO employee VALUES (101, 'Alan', 10000);
INSERT INTO employee VALUES (102, 'Jose', 20000);
INSERT INTO employee VALUES (103, 'Mark', 30000);
COMMIT;

-- anjali tries to select but has no access
SELECT * FROM employee; -- Error: table or view does not exist
```

```
-- anil grants SELECT to anjali
GRANT SELECT ON employee TO anjali;

-- anjali can select now
SELECT * FROM anil.employee;

-- anjali tries to DELETE but has no permission
DELETE FROM anil.employee; -- Error: insufficient privileges

-- anil grants DELETE and UPDATE to anjali
GRANT DELETE, UPDATE ON employee TO anjali;

-- anjali can delete now
DELETE FROM anil.employee;
COMMIT;

-- anil revokes all privileges from anjali
REVOKE SELECT, DELETE, UPDATE ON employee FROM anjali;

-- Disconnect users
DISCONNECT anil;
DISCONNECT anjali;

-- DBA revokes system privileges
REVOKE connect, resource FROM anil, anjali;
```

Useful Queries for Users and Privileges

- List all users in database:

SELECT username FROM all_users;

- Drop user with all objects:

DROP USER anil CASCADE;

DROP USER anjali CASCADE;

Sequences

- Sequence is an object that generates numeric values automatically.
- **Syntax:**

CREATE SEQUENCE <sequence_name> START WITH <start_val> INCREMENT BY <increment_val>;

Example

CREATE SEQUENCE sq1 START WITH 10 INCREMENT BY 10;

CREATE SEQUENCE sq2 START WITH 1 INCREMENT BY 1;

-- Using sequence in insert

INSERT INTO student VALUES (sq1.NEXTVAL, 'raja', 'hyd');

INSERT INTO student VALUES (sq1.NEXTVAL, 'ravi', 'delhi');

INSERT INTO student VALUES (sq1.NEXTVAL, 'ramana', 'vizag');

COMMIT;

-- Get last generated value

SELECT sq1.CURRVAL FROM dual;

Queries related to sequences

- List all sequences:

SELECT sequence_name FROM user_sequences;

- Drop a sequence:

DROP SEQUENCE sq1;

DROP SEQUENCE sq2;

Synonyms

- A synonym is an **alternate name** for a database object (usually a table).
- Allows using a different name to refer to the original object.

Syntax and example

CREATE SYNONYM sy1 FOR employee;

SELECT * FROM sy1;

DELETE FROM sy1;

Queries related to synonyms

- List all synonyms:

SELECT synonym_name FROM user_synonyms;

- Drop a synonym:
- DROP SYNONYM sy1;

DAY 8

Indexes

What is an Index?

- An **Index** is a database object that improves the performance of SELECT queries.
- Like an index in a book, it helps quickly locate data without scanning the entire table.
- Indexes should be created on columns frequently used in the WHERE clause.
- Internally, an index stores two things:
    1. **ROWID** — pointer to the row location
    2. **Index column value** — stored in ascending order

Types of Indexes

1. **Simple Index**
    - Created on a single column.
    - **Syntax:**

CREATE INDEX <index_name> ON <table_name>(column_name);

    - Example:

CREATE INDEX idx1 ON emp(esal);

SELECT * FROM emp WHERE esal = 19000; -- Uses idx1

2. **Complex (Composite) Index**
    - Created on multiple columns.
    - **Syntax:**

CREATE INDEX <index_name> ON <table_name>(col1, col2, ..., colN);

    - Example:

CREATE INDEX idx2 ON emp(eid, ename);

SELECT * FROM emp WHERE eid = 201 AND ename = 'Alan'; -- Uses idx2

Categories of Indexes

1. Non-Unique Index

- Allows duplicate values in the indexed column(s).
- By default, all indexes are non-unique.
- Example:

```
CREATE INDEX idx3 ON emp(deptno);

SELECT * FROM emp WHERE deptno = 10;
```

2. Unique Index

- Ensures all values in the indexed column(s) are unique.
- Example:

```
CREATE UNIQUE INDEX idx4 ON emp(eid);

SELECT * FROM emp WHERE eid = 204;
```

Useful Queries

- List all indexes in the current schema:

```
SELECT index_name FROM user_indexes;
```

- Drop an index:

```
DROP INDEX idx1;

DROP INDEX idx2;

DROP INDEX idx3;

DROP INDEX idx4;
```

DAY 9

Joins

Basic Concepts

- Join is used to retrieve data from one or more tables.
- Joining tables returns related rows based on some condition.

Cartesian Product Example

- SELECT * FROM emp, dept; returns all combinations (6 * 4 = 24 rows if emp has 6 and dept has 4 rows).

Avoid Ambiguous Column Error

- Use **table_name.column_name** format to avoid ambiguity in columns with same names.
- Example:

SELECT emp.eid, emp.ename, emp.esal, dept.dname, dept.dloc

FROM emp, dept;

Table Alias

- A user-defined short name for tables to simplify queries.
- Example:

SELECT e.eid, e.ename, e.esal, d.deptno, d.dname, d.dloc

FROM emp e, dept d;

Types of Joins

1. **Equi-Join**
    - Join based on equality of common columns.
    - Example:

```
SELECT e.eid, e.ename, e.esal, d.dname, d.dloc
FROM emp e, dept d
WHERE e.deptno = d.deptno;
```

2. **Non-Equi Join**
    - Join based on a condition other than equality.
    - Example:

```
SELECT e.eid, e.ename, e.esal, d.dname, d.dloc
FROM emp e, dept d
WHERE e.esal < 35000;
```

3. **Self Join**
    - Joining a table with itself. Requires table aliases.
    - Example:

```
SELECT e1.eid, e1.ename, e1.esal, e2.job, e2.comm
FROM emp e1, emp e2
WHERE e1.deptno = e2.deptno;
```

4. **Cartesian Product**
    - Returns all possible combinations of rows from joined tables (no join condition).
    - Example:

```
SELECT e.eid, e.ename, e.esal, d.dname, d.dloc
FROM emp e, dept d;
```

5) Inner Join

Definition:

- A type of join that returns only matching records between two tables.
- It is similar to **Equi Join** but follows **ANSI SQL standards**.
- **ANSI** stands for *American National Standards Institute*.

Syntax:

SELECT e.eid, e.ename, e.esal, d.dname, d.dloc

FROM emp e INNER JOIN dept d

ON (e.deptno = d.deptno);

Or simply:

sql

CopyEdit

SELECT e.eid, e.ename, e.esal, d.dname, d.dloc

FROM emp e JOIN dept d

ON (e.deptno = d.deptno);

---

6) Outer Join

Definition:

- An extension of Equi Join that returns **matching and non-matching** records.
- + is used as the **SQL outer join operator**.
- **Types of Outer Joins**:
    1. **Left Outer Join**
    2. **Right Outer Join**
    3. **Full Outer Join**

---

i) Left Outer Join

- Returns all records from the **left** table and matched records from the right.

SQL Syntax:

SELECT e.eid, e.ename, e.esal, d.dname, d.dloc

FROM emp e, dept d

WHERE e.deptno = d.deptno(+);

ANSI Syntax:

SELECT e.eid, e.ename, e.esal, d.dname, d.dloc

FROM emp e LEFT OUTER JOIN dept d

ON e.deptno = d.deptno;

---

ii) Right Outer Join

- Returns all records from the **right** table and matched records from the left.

SQL Syntax:

SELECT e.eid, e.ename, e.esal, d.dname, d.dloc

FROM emp e, dept d

WHERE e.deptno(+) = d.deptno;

ANSI Syntax:

SELECT e.eid, e.ename, e.esal, d.dname, d.dloc

FROM emp e RIGHT OUTER JOIN dept d

ON e.deptno = d.deptno;

---

iii) Full Outer Join

- Returns **all records** from both tables, including non-matching rows.

ANSI Syntax:

SELECT e.eid, e.ename, e.esal, d.dname, d.dloc

FROM emp e FULL OUTER JOIN dept d

ON e.deptno = d.deptno;

DAY 10

Views

Definition:

- A **View** is a virtual table based on a query result from one or more tables.
- It **does not store data**; it reflects data from the **base table**.

Syntax:

CREATE VIEW <view_name> AS SELECT ...;

Types of Views:

1. **Simple View**
    - Based on a single table.
    - DML operations are **allowed**.

CREATE VIEW v1 AS SELECT * FROM emp;

2. **Complex View**
    - Based on multiple tables.
    - DML operations are **not allowed**.

CREATE VIEW v2 AS

SELECT e.eid, e.ename, e.esal, d.dname, d.dloc

FROM emp e, dept d

WHERE e.deptno = d.deptno;

3. **With Read Only View**
    - Prevents DML even on simple views.

CREATE VIEW v3 AS SELECT * FROM emp WITH READ ONLY;

4. **With Check Option View**
    - Allows DML **only when condition is satisfied**.

CREATE VIEW v4 AS

SELECT * FROM emp WHERE deptno = 10

WITH CHECK OPTION;

5. **Materialized View**
    - Stores a **physical copy** of data.
    - Requires a **primary key** or **unique key**.

ALTER TABLE emp ADD PRIMARY KEY(eid);

CREATE MATERIALIZED VIEW v5 AS SELECT * FROM emp;

EXEC DBMS_SNAPSHOT.REFRESH('v5');

View Management Queries:

- **List views**:

SELECT view_name FROM user_views;

- **Drop views**:

DROP VIEW v1;

DROP MATERIALIZED VIEW v5;

Subqueries

Definition:

- A **Subquery** is a query embedded inside another query.
- It helps retrieve data using **unknown values**.

Syntax:

SELECT * FROM emp WHERE eid = (SELECT eid FROM emp WHERE ename = 'Alan');

- Executes **inner query first**, then outer.

Types of Subqueries:

1. Single Row Subquery

- Returns **only one row**.

SELECT * FROM emp WHERE eid = (SELECT eid FROM emp WHERE ename = 'Alan');

Interview Example:

Second highest salary

SELECT MAX(esal)

FROM emp

WHERE esal < (SELECT MAX(esal) FROM emp);

Salary greater than Lisa's

SELECT * FROM emp

WHERE esal > (SELECT esal FROM emp WHERE ename = 'Lisa');

---

2) Multiple Row Subquery

A Multiple Row Subquery returns more than one row. These types of subqueries must be handled using multiple row operators.

Multiple Row Operators:

1. ANY – Returns true if any one value from the subquery satisfies the condition.
2. ALL – Returns true if all values from the subquery satisfy the condition.
3. IN – Returns true if the value exists in the result set of the subquery.

Examples:

SELECT * FROM emp WHERE esal > ANY (SELECT esal FROM emp WHERE deptno = 10);

SELECT * FROM emp WHERE esal < ANY (SELECT esal FROM emp WHERE deptno = 10);

SELECT * FROM emp WHERE esal > ALL (SELECT esal FROM emp WHERE deptno = 10);

SELECT * FROM emp WHERE esal IN (SELECT esal FROM emp WHERE deptno = 10);

---

3) Multiple Column Subquery

Used when the subquery returns multiple columns. The number and order of columns in the subquery must match the columns in the main query.

Examples:

```
SELECT * FROM emp

WHERE (eid, ename, esal) IN

(SELECT eid, ename, esal FROM emp WHERE eid = 201);

SELECT eid, ename, esal FROM emp

WHERE (eid, ename, esal) IN

(SELECT eid, ename, esal FROM emp WHERE eid = 201);
```

MERGE Command (UPSERT Operation)

The MERGE command combines INSERT and UPDATE operations. It updates existing rows if matched, otherwise inserts new rows.

Syntax:

```
MERGE INTO target_table USING source_table

ON (condition)

WHEN MATCHED THEN UPDATE SET ...

WHEN NOT MATCHED THEN INSERT (...columns...) VALUES (...values...);
```

Example:

```
MERGE INTO student10 s1

USING student20 s2

ON (s1.sno = s2.sno)

WHEN MATCHED THEN

  UPDATE SET s1.sname = s2.sname, s1.smarks = s2.smarks

WHEN NOT MATCHED THEN

  INSERT (sno, sname, smarks)

  VALUES (s2.sno, s2.sname, s2.smarks);
```

DAY 11

PL/SQL – Procedural Language for SQL

PL/SQL is Oracle's extension to SQL that supports procedural programming constructs.

Key Features:

- Supports loops, conditions, variables, etc.
- Reduces network traffic via block execution
- Enables stored procedures, functions, triggers
- Allows custom exception handling

PL/SQL Block Structure

```
DECLARE
  -- Declaration section (optional)
BEGIN
  -- Executable section (mandatory)
EXCEPTION
  -- Exception handling section (optional)
END;
```

PL/SQL Examples

Print Hello World

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Hello World');
END;
```

Sum of Two Numbers (Hardcoded)

```
DECLARE
  A NUMBER := 10;
  B NUMBER := 20;
  C NUMBER := A + B;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Sum is: ' || C);
END;
```

Sum Using Runtime Input (& substitution)

```
DECLARE
  A NUMBER;
  B NUMBER;
  C NUMBER;
BEGIN
  A := &a;
  B := &b;
  C := A + B;
  DBMS_OUTPUT.PUT_LINE('Sum is: ' || C);
END;
```

DML Operations in PL/SQL

Insert Record

```
DECLARE
 L_Sno NUMBER(3);
 L_Sname VARCHAR2(10);
 L_Sadd VARCHAR2(12);
BEGIN
 L_Sno := &sno;
 L_Sname := '&sname';
 L_Sadd := '&sadd';
 INSERT INTO student VALUES (L_Sno, L_Sname, L_Sadd);
 DBMS_OUTPUT.PUT_LINE('Record Inserted');
END;
```

Update Record

```
DECLARE
 L_Sno NUMBER(3);
BEGIN
 L_Sno := &sno;
 UPDATE student SET sname = 'gogo' WHERE sno = L_Sno;
 DBMS_OUTPUT.PUT_LINE('Record Updated');
END;
```

Delete Record

```
DECLARE
 L_Sno NUMBER(3);
BEGIN
 L_Sno := &sno;
 DELETE FROM student WHERE sno = L_Sno;
 DBMS_OUTPUT.PUT_LINE('Record Deleted');
END;
```

SELECT INTO in PL/SQL

To retrieve data into variables, use SELECT ... INTO ....

Get Employee Name by ID

```
DECLARE
  L_ename VARCHAR2(10);
BEGIN
  SELECT ename INTO L_ename FROM emp WHERE eid = 201;
  DBMS_OUTPUT.PUT_LINE(L_ename);
END;
```

Get Employee Name and Salary by ID

```
DECLARE
  L_ename VARCHAR2(10);
  L_esal NUMBER(10,2);
  L_eid NUMBER(3);
BEGIN
  L_eid := &eid;
  SELECT ename, esal INTO L_ename, L_esal FROM emp WHERE eid = L_eid;
  DBMS_OUTPUT.PUT_LINE(L_ename || ' ' || L_esal);
END;
```

%TYPE and %ROWTYPE Attributes

%TYPE:

Used to declare a variable with the same datatype as a column in a table.

```
L_ename emp.ename%TYPE;
```

%ROWTYPE:

Used to declare a variable to store a full row of data from a table.

```
DECLARE
  A emp%ROWTYPE;
  L_eid emp.eid%TYPE;
BEGIN
  L_eid := &eid;
  SELECT * INTO A FROM emp WHERE eid = L_eid;
  DBMS_OUTPUT.PUT_LINE(A.eid || ' ' || A.ename || ' ' || A.esal || ' ' || A.deptno);
END;
```

Displaying Output in PL/SQL

Before executing any PL/SQL block with output, enable the following:

```
SQL> SET SERVEROUTPUT ON;
```

DAY 12

◈ Control Statements in PL/SQL

1. IF THEN

Executes statements only when the condition is TRUE.

```
DECLARE
  A NUMBER := 5;
BEGIN
  IF A > 2 THEN
    DBMS_OUTPUT.PUT_LINE('Welcome');
  END IF;
END;
/
```

2. IF THEN ELSE

Handles both TRUE and FALSE scenarios.

```
DECLARE
  A NUMBER := 25;
BEGIN
  IF A < 18 THEN
    DBMS_OUTPUT.PUT_LINE('You are not eligible to vote');
  ELSE
    DBMS_OUTPUT.PUT_LINE('You are eligible to vote');
  END IF;
END;
/
```

3. IF THEN ELSIF THEN ELSE

Used for multiple condition checks.

```
DECLARE
```

```
 n NUMBER(3);
BEGIN
 n := &opt;
 IF n = 100 THEN
  DBMS_OUTPUT.PUT_LINE('It is police number');
 ELSIF n = 103 THEN
  DBMS_OUTPUT.PUT_LINE('It is enquiry number');
 ELSIF n = 108 THEN
  DBMS_OUTPUT.PUT_LINE('It is emergency number');
 ELSE
  DBMS_OUTPUT.PUT_LINE('Invalid option');
 END IF;
END;
/
```

🔷 Loops in PL/SQL

1. Simple Loop

Repeats code until explicitly exited.

```
DECLARE
 A NUMBER := 1;
BEGIN
 LOOP
  DBMS_OUTPUT.PUT_LINE(A);
  EXIT WHEN A = 10;
  A := A + 1;
 END LOOP;
END;
/
```

2. WHILE Loop

Executes while the condition is TRUE.

```
DECLARE
  A NUMBER := 10;
BEGIN
  WHILE A >= 1 LOOP
    DBMS_OUTPUT.PUT_LINE(A);
    A := A - 1;
  END LOOP;
END;
/
```

3. FOR Loop

Predefined iteration range.

```
DECLARE
  A NUMBER;
BEGIN
  FOR A IN 1..10 LOOP
    DBMS_OUTPUT.PUT_LINE('5 * ' || A || ' = ' || 5*A);
  END LOOP;
END;
/
```

DAY 13

◈ Exceptions in PL/SQL

➤ Types:

1. **Predefined Exceptions** – Built-in
2. **User-defined Exceptions** – Defined by the developer

---

1. Predefined Exceptions

a) NO_DATA_FOUND

Occurs when a SELECT INTO query returns no rows.

```
BEGIN
  SELECT ename INTO L_ename FROM emp WHERE eid = 209;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Employee not found');
END;
/
```

b) TOO_MANY_ROWS

Occurs when SELECT INTO returns more than one row.

```
BEGIN
  SELECT ename INTO L_ename FROM emp WHERE deptno = 10;
EXCEPTION
  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('More than one row found');
END;
/
```

c) ZERO_DIVIDE

Occurs when a number is divided by zero.

```
BEGIN
```

```
  A := 10 / 0;
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');
END;
/
```

d) VALUE_ERROR

Occurs due to data type mismatch or value exceeding limits.

```
DECLARE
  A NUMBER(3);
BEGIN
  A := 12345;
EXCEPTION
  WHEN VALUE_ERROR THEN
    DBMS_OUTPUT.PUT_LINE('Value out of range');
END;
/
```

e) DUP_VAL_ON_INDEX

Occurs when inserting a duplicate value into a primary key.

```
BEGIN
  INSERT INTO emp VALUES (202, 'Chan', 34000, 50, 'Salesman', 100);
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Duplicate ID not allowed');
    END;
/
```

f) OTHERS

Catches all other unhandled exceptions.

```
BEGIN
  SELECT ename INTO L_ename FROM emp WHERE eid = 209;
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred');
END;
/
```

---

2. User-Defined Exceptions

Steps:

1. Declare an exception.
2. Raise the exception using RAISE.
3. Handle it using WHEN.

Example:

```
DECLARE
  SAL NUMBER := 5000;
  MY_EXP EXCEPTION;
BEGIN
  IF SAL > 2000 THEN
    RAISE MY_EXP;
  END IF;
EXCEPTION
  WHEN MY_EXP THEN
    DBMS_OUTPUT.PUT_LINE('Salary is too high');
END;
/
```

DAY 14

❖ Cursors in PL/SQL

➤ Types:

1. **Implicit Cursor** – Managed automatically by Oracle.
2. **Explicit Cursor** – Managed manually by the developer.

Implicit Cursor Attributes

Attribute	Description
SQL%ISOPEN	Always FALSE for implicit cursors
SQL%FOUND	TRUE if at least one row affected
SQL%NOTFOUND	Opposite of SQL%FOUND
SQL%ROWCOUNT	Returns number of rows affected

Example: SQL%FOUND

```
BEGIN
  UPDATE emp SET ename = 'AntMan' WHERE eid = 201;
  IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Record updated');
  ELSE
    DBMS_OUTPUT.PUT_LINE('No records found');
  END IF;
END;
/
```

Example: SQL%ROWCOUNT

```
BEGIN
```

```
  DELETE FROM student;
  DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' Records Deleted');
END;
/
```

PL/SQL Essentials Reference Sheet

🔷 1. Output in PL/SQL

- Use SET SERVEROUTPUT ON to view output.

```
SQL> SET SERVEROUTPUT ON;
```

---

2. Explicit Cursors

- When SELECT returns more than one row.
- Cursor Attributes:
    - %ISOPEN, %FOUND, %NOTFOUND, %ROWCOUNT

Example 1: Fetch ename and esal
Example 2: Fetch full employee row using %ROWTYPE

---

3. PL/SQL Procedures

- Created using CREATE OR REPLACE PROCEDURE
- Types of Parameters: IN, OUT, IN OUT

Examples:

- Hello World Procedure
- Sum of two numbers using IN
- Return sum using OUT
- Square using IN OUT
- DML Example: Delete student by sno

Executing Procedures

sql

CopyEdit

EXEC procedure_name(params...);

---

4. PL/SQL Functions

- Must return a value
- Can be called in SELECT statements

Examples:

- Add two numbers
- Calculate 10% TDS from salary

Important Notes:

- No DML allowed inside functions
- Called as: SELECT function_name(...) FROM DUAL;

---

5. Procedures vs Functions

Feature	Procedure	Function
Return Value	Optional	Must return a value
DML Operations	Allowed	Not allowed
Call in SELECT	Not allowed	Allowed

---

6. Useful Queries

- List all procedures:

SELECT object_name FROM user_objects WHERE object_type='PROCEDURE';

- View source code:

SELECT text FROM user_source WHERE name='P1'; -- use uppercase

---

🔷 7. Packages

- **Specification**: Declare procedures/functions
- **Body**: Define logic

Example:

```
-- Package Spec
CREATE OR REPLACE PACKAGE pkg1 IS
  PROCEDURE p1(A IN NUMBER, B IN NUMBER);
END pkg1;
/

-- Package Body
CREATE OR REPLACE PACKAGE BODY pkg1 IS
  PROCEDURE p1(A IN NUMBER, B IN NUMBER) IS
    C NUMBER;
  BEGIN
    C := A + B;
    DBMS_OUTPUT.PUT_LINE(C);
  END;
END pkg1;
/
```

🔷 8. Triggers

- Trigger Types:
    - **Statement-level** (default)

- **Row-level** (FOR EACH ROW)

- Events:
  - INSERT, UPDATE, DELETE
- Timings:
  - BEFORE, AFTER, INSTEAD OF

Example:

```
CREATE OR REPLACE TRIGGER trg1
BEFORE DELETE ON student
BEGIN
  DBMS_OUTPUT.PUT_LINE('Yahoo! You deleted');
END;
/
```

◈ 9. Dropping Objects

```
-- Procedures
DROP PROCEDURE procedure_name;

-- Functions
DROP FUNCTION function_name;

-- Packages
DROP PACKAGE package_name;
DROP PACKAGE BODY package_name;

-- Triggers
DROP TRIGGER trigger_name;
```